

Compiler Register Allocation

01
10

DA Programming Project II

Group T9G2 / Spring 2026



Alexandre Dinis Alves Teixeira
Carlos Francisco de Sousa Ferreira Magalhaes
Diogo
Rodrigo Martins Dias

`make test`

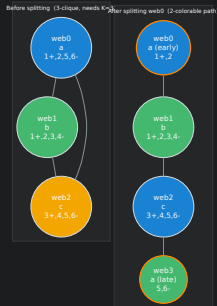
`make docs`

`./register_alloc -b ...`

Pipeline

Core flow

- parse ranges + config
- fuse ranges into webs
- build $\text{Graph}<\text{int}>$ interference graph
- color with at most k registers
- export allocation text + colored DOT



Course $\text{Graph}<\text{int}>$ as primary representation. Web ids are vertex labels.

Spilling and Splitting

01
10

Bounded spilling

Colored Register Allocation
2 register(s), 1 memory web(s)

spills: web0

Legend

orange border = split-derived

M = memory

same fill = same register

web0
a
M
1+,2,3-

web1
b
r1
1+,2,3-

web2
c
r0
1+,2,3-

Bounded splitting

Colored Register Allocation
2 register(s), 0 memory web(s)

splits: web0 -> web0,web3

Legend

orange border = split-derived

M = memory

same fill = same register

web0
a
r0
1+,2

web1
b
r1
1+,2,3,4-

web2
c
r0
3+,4,5,6-

web3
a
r2
5,6

Splitting preserves original markers. 1+,2,5,6- splits into 1+,2 and 5,6- — no fabricated 2- or 5+.

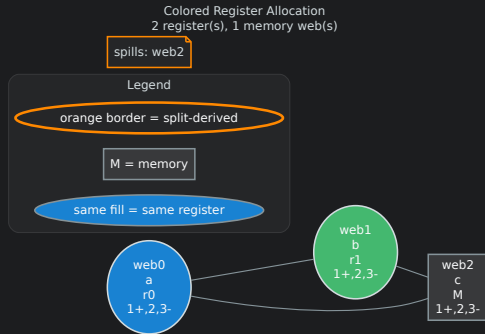
Metadata: spills: N / splits: N.

Custom Free Algorithm

01
10

Hybrid strategy

- edgeless graphs: 1 register
- complete graphs: K colored, rest spilled
- bipartite graphs: BFS 2-coloring
- general: DSatur greedy coloring
- small/medium: branch-and-bound refines spills



Invariant: interfering colored webs never share a register. Memory only for uncolored webs.

Demo and Testing

Inputs for the demo

- `complex_allocation.txt` + `free4.txt`: dense allocator stress test
- `splitting_showcase.txt` + `splitting2.txt`: split reduces K from 3 to 2
- `spilling5.txt`: bounded spilling in a dense graph

```
./register_alloc -b  
  inputs/advanced/ranges/  
complex_allocation.txt  
  inputs/advanced/registers/free4.txt  
  alloc.txt
```

`make test` — 76 unit + 9 integration (passes clean). DOT graphs written alongside allocation output.

Complexities

Parse: $O(LP)$ | Build webs: $O(VR^2P \log P)$

Build graph: $O(W^2P + EW)$

Basic: $O(W(W^2 + E))$

Spilling: $O((S + 1)W(W^2 + E))$

Splitting: $O(SQ(W^2P + EW + W(W^2 + E)))$

Free: $O(W^2 + E \log W)$ heuristic; exact pass guarded

W = webs, E = adjacency entries, P = program points, R = ranges/variable, S = recovery bound, Q = candidate split positions. Bounds include vector-backed `Graph<int> findVertex` cost. Doxygen: [docs/html/index.html](https://docs.html/index.html).